

PCA and LDA

COMP3314
Introduction to Machine Learning

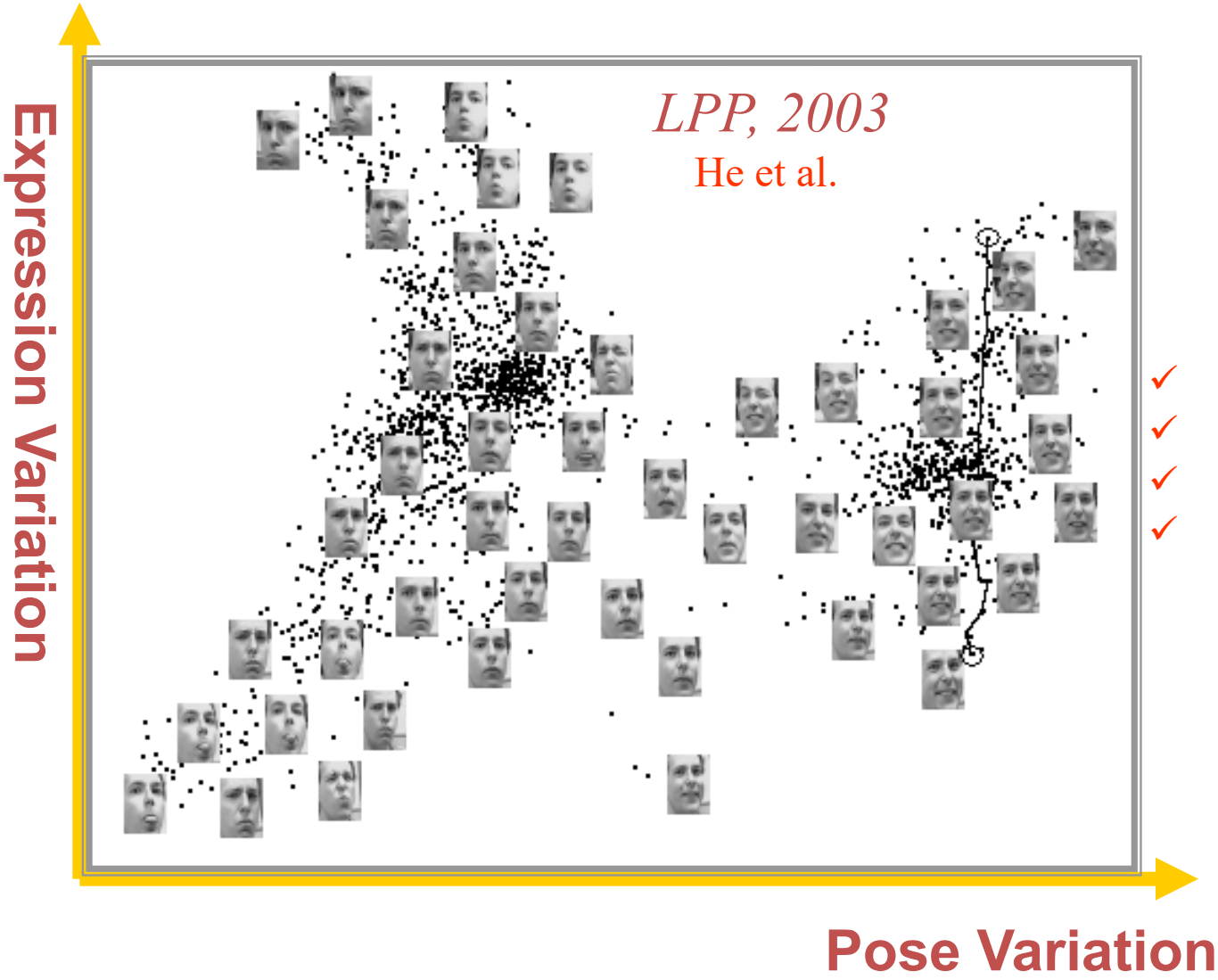
Dong Xu

The space of all face images

- When viewed as vectors of pixel values, face images are extremely high-dimensional
  - 100x100 image = 10,000 dimensions
  - Slow and lots of storage
- But very few 10,000-dimensional vectors are valid face images
- We want to effectively model the subspace of face images



Why Conduct Dimensionality Reduction?



- ✓ **Visualization**
- ✓ **Feature Extraction**
- ✓ **Computation Efficiency**
- ✓ **Broad Applications**

Face Recognition
Human Gait Recognition
CBIR

Uncover intrinsic structure

Dimension reduction

- Two approaches are available to perform dimensionality reduction
  - **Feature extraction**: creating a subset of new features by combinations of the existing features
  - **Feature selection**: choosing a subset of all the features (the ones more informative)

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{feature selection}} \begin{bmatrix} x_{i_1} \\ x_{i_2} \\ \vdots \\ x_{i_M} \end{bmatrix} \qquad \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{feature extraction}} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = f\left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \right)$$

- The problem of feature extraction can be stated as
  - Given a feature space $\boldsymbol{x_i} \in R^N$ find a mapping **y=f(x):** $R^N \rightarrow R^M$ with $M<N$ such that the transformed feature vector $\boldsymbol{y_i} \in R^M$ preserves (most of) the information or structure in R^N.
  - An **optimal** mapping ***y*=f(*x*)** will be one that results in **no increase in the minimum probability of error**
    - This is, a Bayes decision rule applied to the initial space R^N and to the reduced space R^M yield the same classification rate

Dimension reduction

- In general, the optimal mapping $y=f(x)$ will be a non-linear function
  - However, there is no systematic way to generate non-linear transforms
    - The selection of a particular subset of transforms is problem dependent
  - For this reason, feature extraction is commonly limited to linear transforms: ***y=Wx***
    - This is, *y* is a linear projection of *x*
    - NOTE: When the mapping is a non-linear function, the reduced space is called a **manifold**

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \xrightarrow{\text{linear feature extraction}} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1N} \\ w_{21} & w_{21} & \cdots & w_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ w_{M1} & w_{M2} & \cdots & w_{MN} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$$

- We will focus on linear feature extraction.

Principal Component Analysis (PCA)

- Given: N data points $\mathbf{x}_1, \ldots, \mathbf{x}_N$ in R^d

- We want to find a new set of features that are linear combinations of original ones:

$$u(\mathbf{x}_i) = \mathbf{u}^T(\mathbf{x}_i - \boldsymbol{\mu})$$

  ($\boldsymbol{\mu}$: mean of data points)

- Choose unit vector $\mathbf{u}$ in R^d that captures the most data variance

Principal Component Analysis (PCA)

- Direction that maximizes the variance of the projected data:

Maximize $\frac{1}{N}\sum_{i=1}^{N} \underbrace{\mathbf{u}^{\mathrm{T}}(\mathbf{x}_i - \mu)}_{\text{Projection of data point}}(\mathbf{u}^{\mathrm{T}}(\mathbf{x}_i - \mu))^{\mathrm{T}}$ subject to ||**u**||=1

$$= \mathbf{u}^{\mathrm{T}} \underbrace{\left[1/N \sum_{i=1}^{N} (\mathbf{x}_i - \mu)(\mathbf{x}_i - \mu)^{\mathrm{T}}\right]}_{\text{Covariance matrix of data}} \mathbf{u}$$

$$= \mathbf{u}^{\mathrm{T}} \Sigma \mathbf{u}$$

The direction that maximizes the variance is the eigenvector associated with the largest eigenvalue of Σ

Implementation issue

- Covariance matrix is huge ($d=M^2$)
- But typically # examples << d
- Simple trick
  - **X** is dxN matrix of normalized training data
  - Solve for eigenvectors **u** of $\mathbf{X}^T\mathbf{X}$ instead of $\mathbf{X}\mathbf{X}^T$
  - Then **Xu** is eigenvector of covariance $\mathbf{X}\mathbf{X}^T$
  - Need to normalize each vector of **Xu** into unit length

Eigenfaces (PCA on face images)

1. Compute the principal components ("eigenfaces") of the covariance matrix

$$\boldsymbol{X} = [(\boldsymbol{x_1} - \boldsymbol{\mu}), (\boldsymbol{x_2} - \boldsymbol{\mu}), \ldots \quad , (\boldsymbol{x_N} - \boldsymbol{\mu})]$$
$$[\boldsymbol{U}, \boldsymbol{\lambda}] = \mathrm{eig}(\boldsymbol{X}^{\boldsymbol{T}}\boldsymbol{X})$$
$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{U}$$

2. Keep K eigenvectors with largest eigenvalues

$$\boldsymbol{V} = \boldsymbol{V}(:, \mathrm{largest_eig})$$

3. Represent all face images in the dataset as linear combinations of eigenfaces
   - Perform nearest neighbor on these coefficients

$$\boldsymbol{X_{pca}} = \boldsymbol{V}\left(:, \mathrm{largest_{eig}}\right)^{\boldsymbol{T}}\boldsymbol{X}$$

M. Turk and A. Pentland, Face Recognition using Eigenfaces, CVPR 1991

Eigenfaces example

- Training images
- $\mathbf{x}_1,\ldots,\mathbf{x}_N$

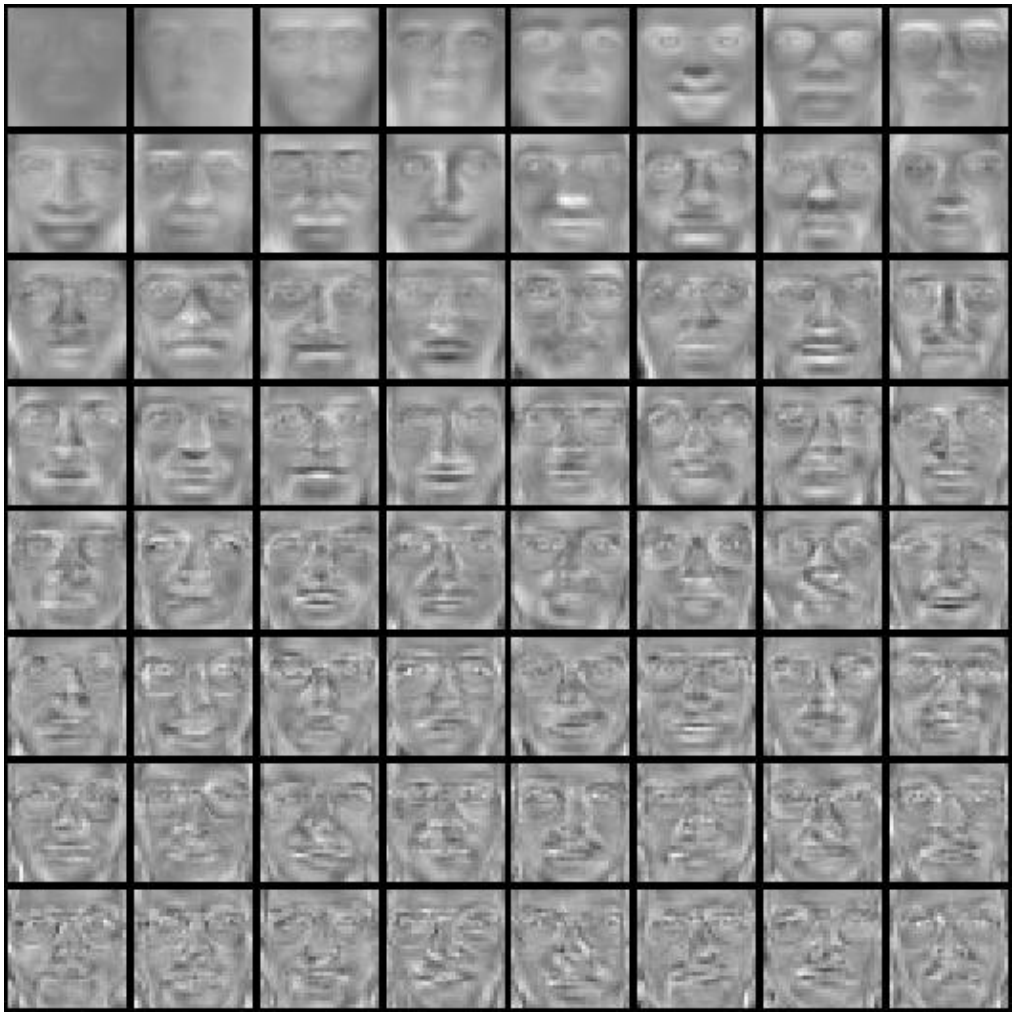


Eigenfaces example

Mean: μ



Top eigenvectors: $u_1,\ldots u_k$



Representation and reconstruction

- Face **x** in "face space" coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^{\mathrm{T}}(\mathbf{x} - \mu), \ldots, \mathbf{u}_k^{\mathrm{T}}(\mathbf{x} - \mu)]$$

$$= \quad w_1, \ldots, w_k$$

Representation and reconstruction

- Face **x** in "face space" coordinates:



$$\mathbf{x} \rightarrow [\mathbf{u}_1^{\mathrm{T}}(\mathbf{x} - \mu), \ldots, \mathbf{u}_k^{\mathrm{T}}(\mathbf{x} - \mu)]$$
$$= \quad w_1, \ldots, w_k$$

- Reconstruction:



$\hat{x}$ = μ + $w_1u_1+w_2u_2+w_3u_3+w_4u_4+$ …

Reconstruction

P = 4

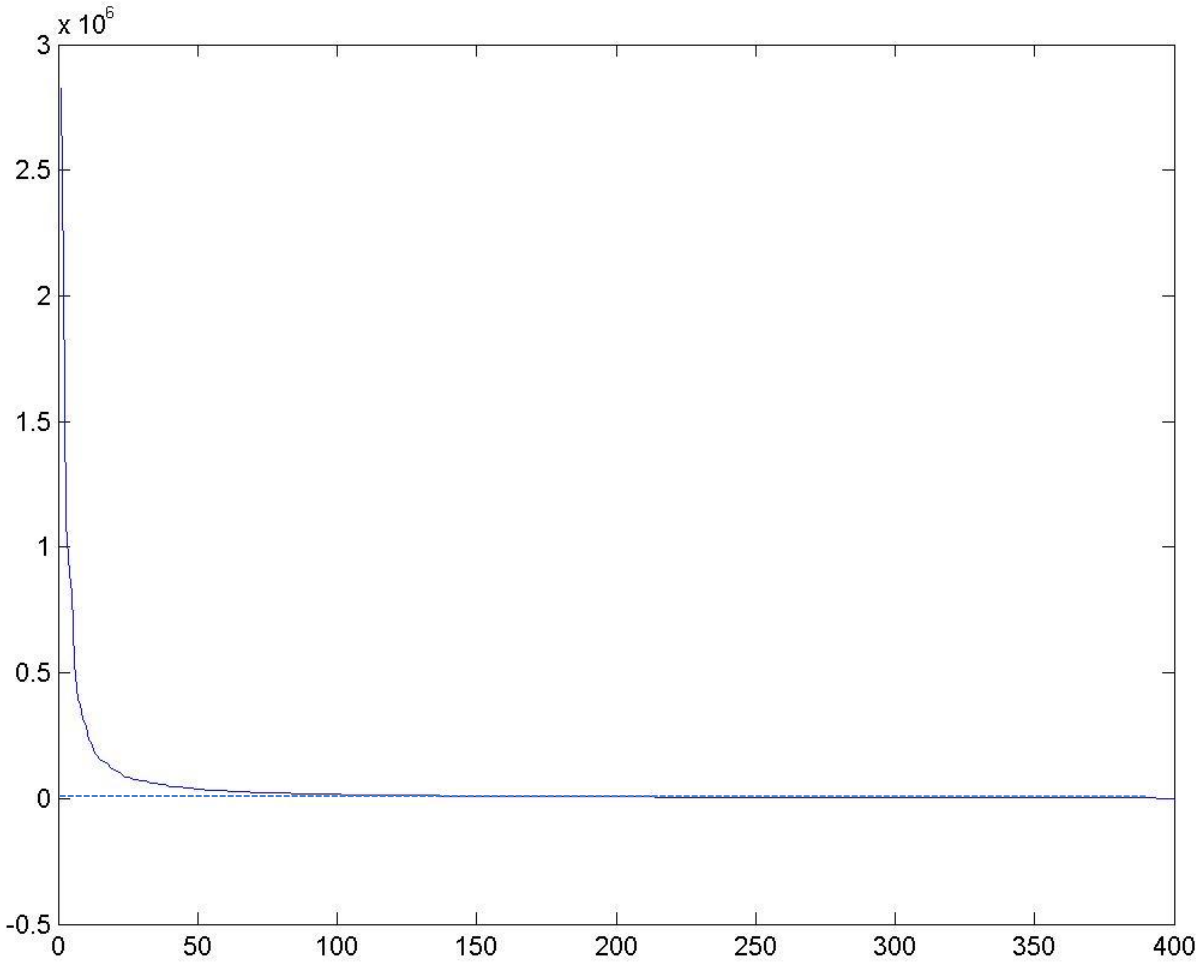
P = 200

P = 399



After computing eigenfaces using 399 face images from ORL face database

Eigenvalues (variance along eigenvectors)



- The x-axis lists principal components in descending order of eigenvalue.
- Each eigenvalue equals the variance captured along its principal-component direction.
- A fast decay means that a small number of components preserves most of the variance.

PCA: An Example

- Compute the principal components for the following two-dimensional dataset
  - **X=(x1,x2)={(1,2),(3,3),(3,5),(5,4),(5,6),(6,5),(8,7),(9,8)}**
    - Let's first plot the data to get an idea of which solution we should expect
- SOLUTION (by hand)
  - The (biased) covariance estimate of the data is:

$$\Sigma_x = \begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}$$

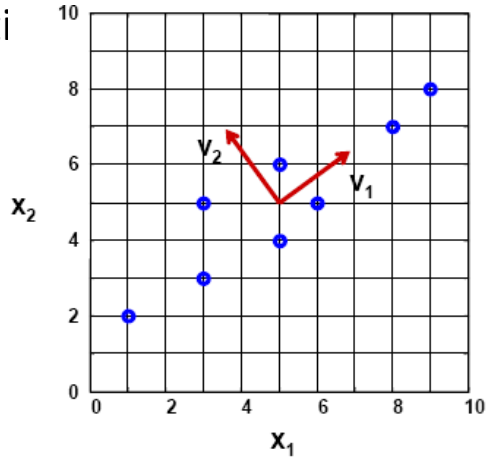
  - The eigenvalues are the zeros of the characteristic equati

$$\Sigma_x v = \lambda v \Rightarrow |\Sigma_x - \lambda I| = 0 \Rightarrow \begin{vmatrix} 6.25-\lambda & 4.25 \\ 4.25 & 3.5-\lambda \end{vmatrix} = 0 \Rightarrow \lambda_1 = 9.34;\ \lambda_2 = 0.41;$$

  - The eigenvectors are the solutions of the system

$$\begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}\begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} \lambda_1 v_{11} \\ \lambda_1 v_{12} \end{bmatrix} \Rightarrow \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0.81 \\ 0.59 \end{bmatrix}$$

$$\begin{bmatrix} 6.25 & 4.25 \\ 4.25 & 3.5 \end{bmatrix}\begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} \lambda_2 v_{21} \\ \lambda_2 v_{22} \end{bmatrix} \Rightarrow \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} -0.59 \\ 0.81 \end{bmatrix}$$



Dimension Reduction and Reconstruction

- Let us look at the first data point (1,2)
- Dimension reduction using the projection matrix

$$(0.81 \quad 0.59)\begin{pmatrix}1\\2\end{pmatrix} = 1.99,$$ first dimension using the leading eigenvector

$$(-0.59 \quad 0.81)\begin{pmatrix}1\\2\end{pmatrix} = 1.03,$$ second dimension using the second eigenvector

- Reconstruction
  - Reconstruction using the leading eigenvector only

$$1.99\begin{pmatrix}0.81\\0.59\end{pmatrix} = \begin{pmatrix}1.6119\\1.1741\end{pmatrix}$$

  - Reconstruction using both eigenvectors

$$1.99\begin{pmatrix}0.81\\0.59\end{pmatrix} + 1.03\begin{pmatrix}-0.59\\0.81\end{pmatrix} = \begin{pmatrix}1\\2\end{pmatrix}$$

Recognition with eigenfaces

Process labeled training images

- Find mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$
- Find k principal components (eigenvectors of $\boldsymbol{\Sigma}$) $\mathbf{u}_1,\ldots\mathbf{u}_k$
- Project each training image $\mathbf{x}_i$ onto subspace spanned by principal components:
  $(w_{i1},\ldots,w_{ik}) = (\mathbf{u}_1^T(\mathbf{x}_i - \boldsymbol{\mu}), \ldots, \mathbf{u}_k^T(\mathbf{x}_i - \boldsymbol{\mu}))$

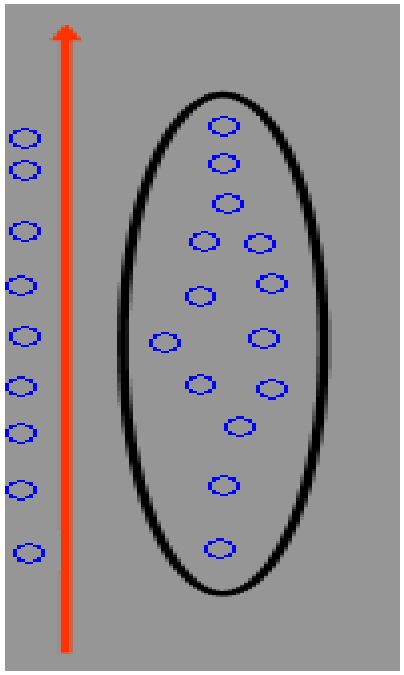
Given novel image $\mathbf{x}$

- Project onto subspace:
  $(w_1,\ldots,w_k) = (\mathbf{u}_1^T(\mathbf{x} - \boldsymbol{\mu}), \ldots, \mathbf{u}_k^T(\mathbf{x} - \boldsymbol{\mu}))$
- Optional: check reconstruction error $\mathbf{x} - \hat{\mathbf{x}}$ to determine whether image is really a face
- Classify as closest training face in k-dimensional subspace

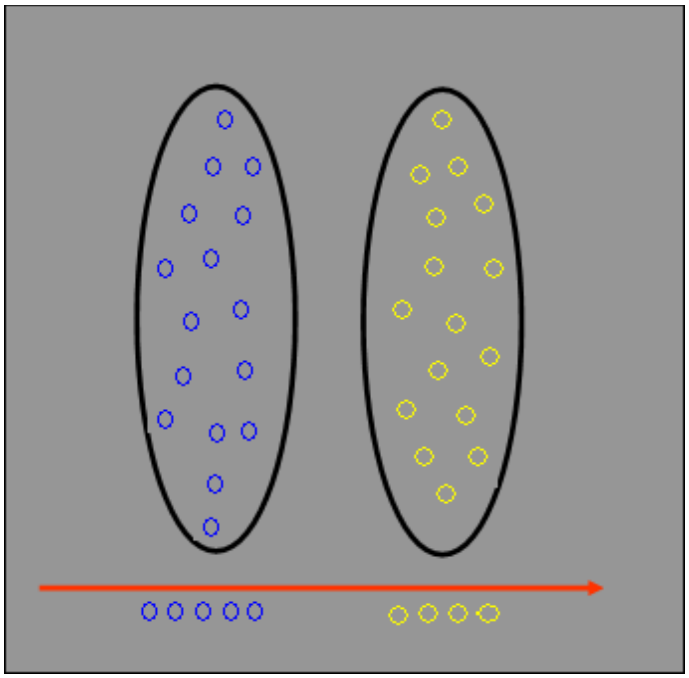
M. Turk and A. Pentland, Face Recognition using Eigenfaces, CVPR 1991

PCA: Limitations

- The direction of maximum variance is not always good for classification



PCA



LDA

Examples: 2D space to 1D space

A more discriminative subspace: FLD

- Fisher Linear Discriminants → "Fisher Faces"

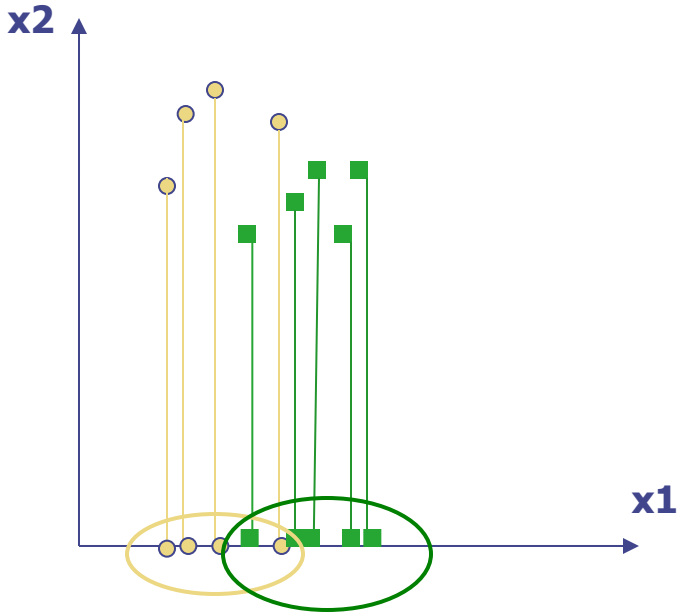
- PCA preserves maximum variance

- FLD preserves discrimination
  - Find projection that maximizes scatter between classes and minimizes scatter within classes

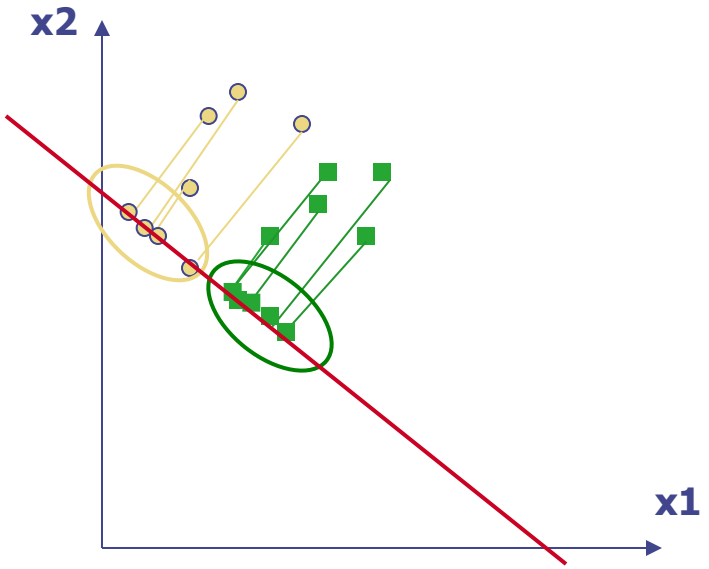
Reference: Eigenfaces vs. Fisherfaces, Belheumer et al., PAMI 1997

Illustration of the Projection

- Using two classes as example:

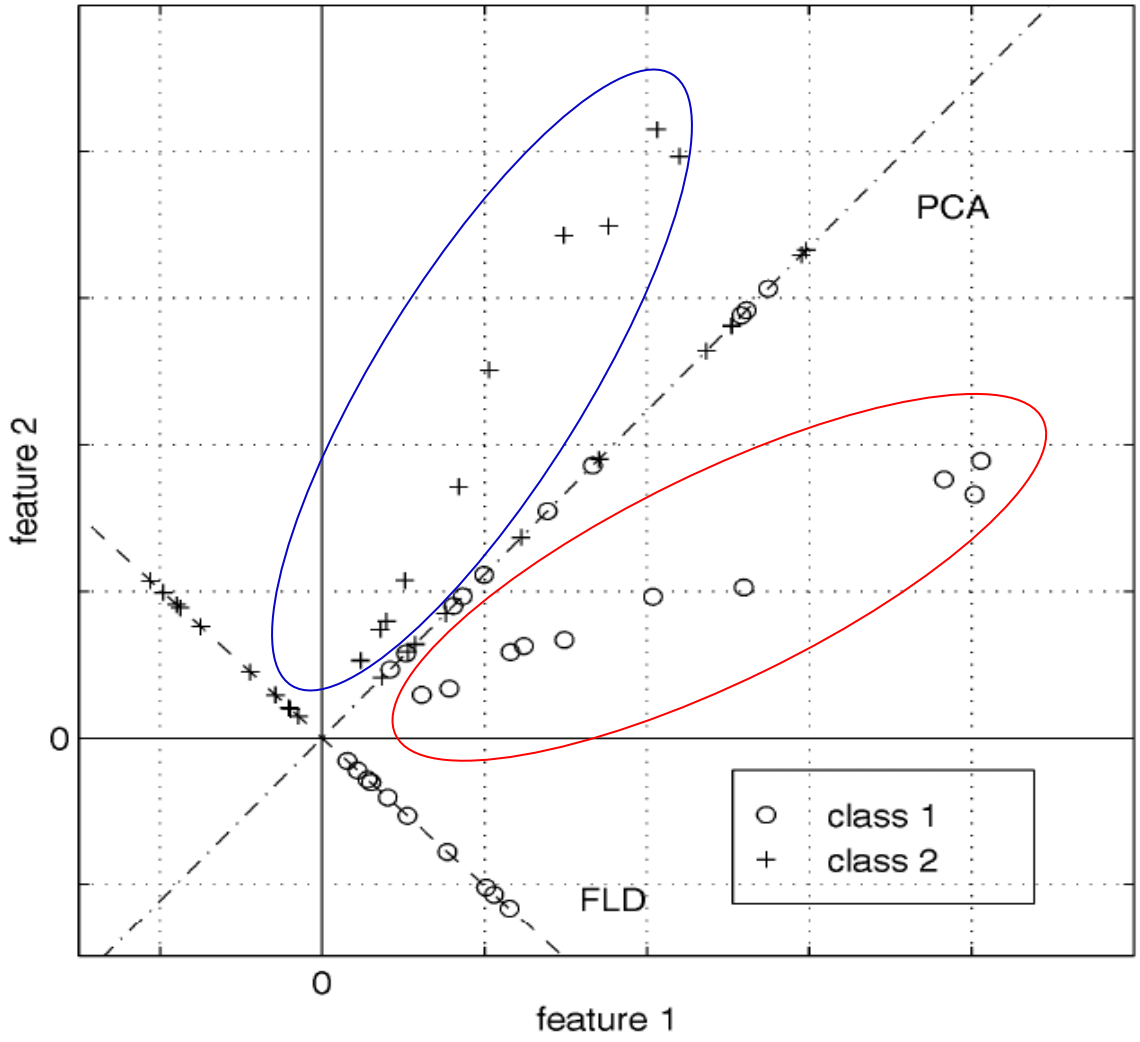


Poor Projection

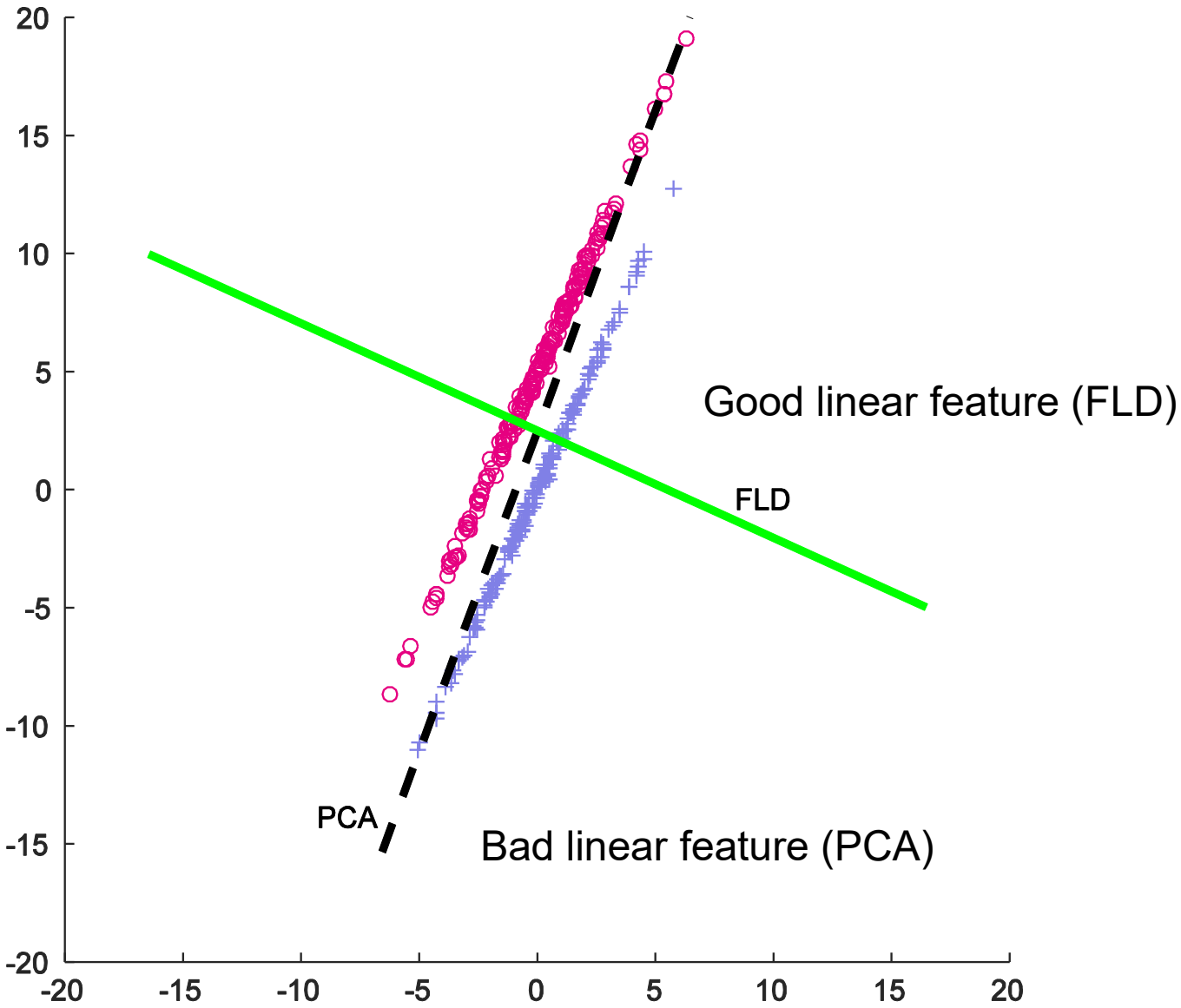


Good

Comparing with PCA



Idea: motivation of FLD



Mathematical formulation

- Two categories $y_i \in \{1,2\}$, data $\boldsymbol{x}_i$, N_1, N_2 examples in each category
- Hope to find a projection direction, $u = \boldsymbol{w}^T \boldsymbol{x}$, such that after the projection, examples from two categories are separate

- The means of two categories are
  - $\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{y_i=1} \boldsymbol{x}_i$,
  - $\boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{y_i=2} \boldsymbol{x}_i$
  - The means after projection are $m_1 = \boldsymbol{w}^T \boldsymbol{\mu}_1$, $m_2 = \boldsymbol{w}^T \boldsymbol{\mu}_2$

Objective: Fisher's Criterion

- How to describe "level of separation"?
- Maximize $(m_2 - m_1)^2$?
  - This value can be infinitely large. How?
  - Add a constraint $\boldsymbol{w}^T\boldsymbol{w} = 1$
  - But, a large value does not mean a better direction. How?
- Fisher's criterion
  - While requiring $|m_2 - m_1|$ to be maximized, also asking the two categories to concentrate rather than dispersed after projection:

$$J(\boldsymbol{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2}$$

Measuring separation

- For 1D data, the natural measure is variance ($k = 1,2$)

$$s_k^2 = \sum_{y_i=k} (u_i - m_k)^2$$

  - Called the within class scatter
- $s_1^2 + s_2^2$
  - total within-class scatter
- $s_k^2 = \sum_{y_i=k} (u_i - m_k)^2 = \sum_{y_i=k} \left(\boldsymbol{w}^T (\boldsymbol{x}_i - \boldsymbol{\mu}_k)\right)^2 = \boldsymbol{w}^T \sum_{y_i=k} (\boldsymbol{x}_i - \boldsymbol{\mu}_k)(\boldsymbol{x}_i - \boldsymbol{\mu}_k)^T \boldsymbol{w}$
- $(m_2 - m_1)^2 = \boldsymbol{w}^T (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \boldsymbol{w}$

Scatter matrix

- $S_k = \sum_{y_i=k}(\boldsymbol{x}_i - \boldsymbol{\mu}_k)(\boldsymbol{x}_i - \boldsymbol{\mu}_k)^T$
- Within-class scatter matrix

$$S_W = S_1 + S_2$$

- Between-class scatter matrix

$$S_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T$$

- Matrix format for Fisher's criterion
  - $\max J(\boldsymbol{w}) = \frac{\boldsymbol{w}^T S_B \boldsymbol{w}}{\boldsymbol{w}^T S_W \boldsymbol{w}},\ s.t. \boldsymbol{w}^T \boldsymbol{w} = 1$
  - This optimization problem is called a generalized Rayleigh quotient

Optimization: How to solve it?

- (Simplification/transformation) Exercise: Use the method of Lagrange multipliers to show that the optimal solutions must satisfy

$$S_B \boldsymbol{w} = \lambda S_W \boldsymbol{w}$$

- This is the generalized eigenvalue problem
  - Obtain generalized eigenvalues and eigenvectors of S_B and S_W
- In fact, we do not need to do that
  - $S_B \boldsymbol{w} = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \boldsymbol{w} \propto (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$
  - $(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) = \lambda S_W \boldsymbol{w}$!

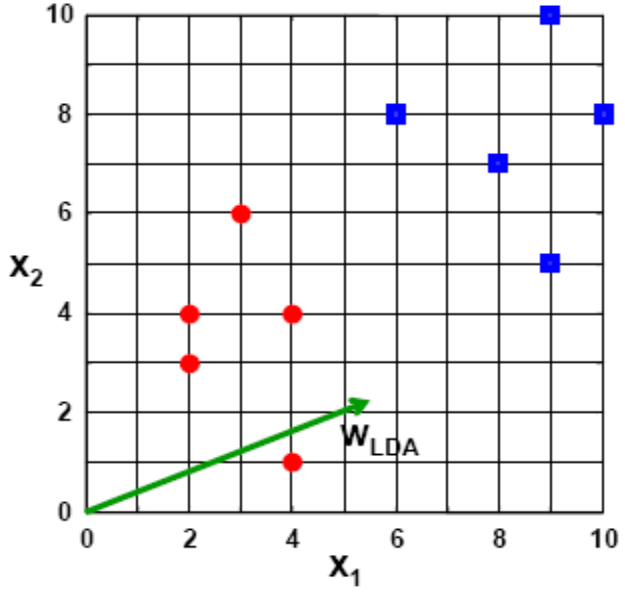
FLD

1. Compute $\boldsymbol{\mu}_2, \boldsymbol{\mu}_1$
2. Compute S_W
3. Compute $\boldsymbol{w} = S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$
4. Normalize:

$$\boldsymbol{w} \leftarrow \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|}$$

LDA Example

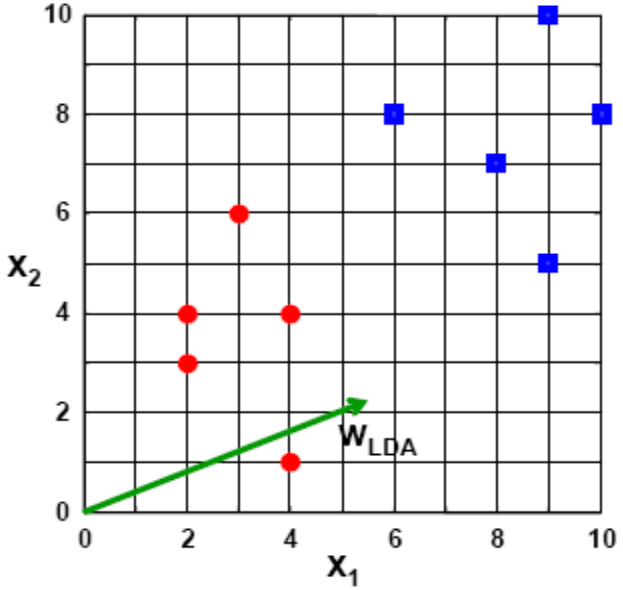
- Compute the Linear Discriminant projection for the following two-dimensional dataset
- $\mathcal{X}_1 = \{(4,1), (2,4), (2,3), (3,6), (4,4)\}$,
  $\mathcal{X}_2 = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$.



- **SOLUTION**
- The class statistics are:
  - $\boldsymbol{\mu}_1 = \begin{bmatrix} 3.00 \\ 3.60 \end{bmatrix}, \qquad \boldsymbol{\mu}_2 = \begin{bmatrix} 8.40 \\ 7.60 \end{bmatrix}.$
  - $S_1 = \sum_{\boldsymbol{x}_i \in \mathcal{X}_1} (\boldsymbol{x}_i - \boldsymbol{\mu}_1)\,(\boldsymbol{x}_i - \boldsymbol{\mu}_1)^T = \begin{bmatrix} 4.00 & -2.00 \\ -2.00 & 13.20 \end{bmatrix},$
  - $S_2 = \sum_{\boldsymbol{x}_i \in \mathcal{X}_2} (\boldsymbol{x}_i - \boldsymbol{\mu}_2)\,(\boldsymbol{x}_i - \boldsymbol{\mu}_2)^T = \begin{bmatrix} 9.20 & -0.20 \\ -0.20 & 13.20 \end{bmatrix}.$
- The within- and between-class scatter are:
  - $S_W = S_1 + S_2 = \begin{bmatrix} 13.20 & -2.20 \\ -2.20 & 26.40 \end{bmatrix}.$
  - $S_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T = \begin{bmatrix} 29.16 & 21.60 \\ 21.60 & 16.00 \end{bmatrix}.$

LDA Example

- Compute the Linear Discriminant projection for the following two-dimensional dataset
- $\mathcal{X}_1 = \{(4,1), (2,4), (2,3), (3,6), (4,4)\}$,
  $\mathcal{X}_2 = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$.



- **SOLUTION**
  - $S_W = \begin{bmatrix} 13.20 & -2.20 \\ -2.20 & 26.40 \end{bmatrix}$, $S_B = \begin{bmatrix} 29.16 & 21.60 \\ 21.60 & 16.00 \end{bmatrix}$.
- The LDA projection is then obtained as the solution of the generalized eigenvalue problem
  - $S_W^{-1} S_B \boldsymbol{v} = \lambda \boldsymbol{v}$, $S_W^{-1} S_B \approx \begin{bmatrix} 2.38 & 1.76 \\ 1.02 & 0.75 \end{bmatrix}$.
  - $\det\left(S_W^{-1} S_B - \lambda I\right) = \begin{vmatrix} 2.38 - \lambda & 1.76 \\ 1.02 & 0.75 - \lambda \end{vmatrix} \approx 0.$
  - $\lambda_1 \approx 3.13$, $\lambda_2 \approx 0$, $\boldsymbol{v}_1 \approx \begin{bmatrix} 0.92 \\ 0.39 \end{bmatrix}$.
- Or directly by
  - $\tilde{\boldsymbol{w}} = S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \approx \begin{bmatrix} 0.44 \\ 0.19 \end{bmatrix}$, $\hat{\boldsymbol{w}} = \frac{\tilde{\boldsymbol{w}}}{\|\tilde{\boldsymbol{w}}\|} \approx \begin{bmatrix} 0.92 \\ 0.39 \end{bmatrix}$.

Results: Eigenface vs. Fisherface

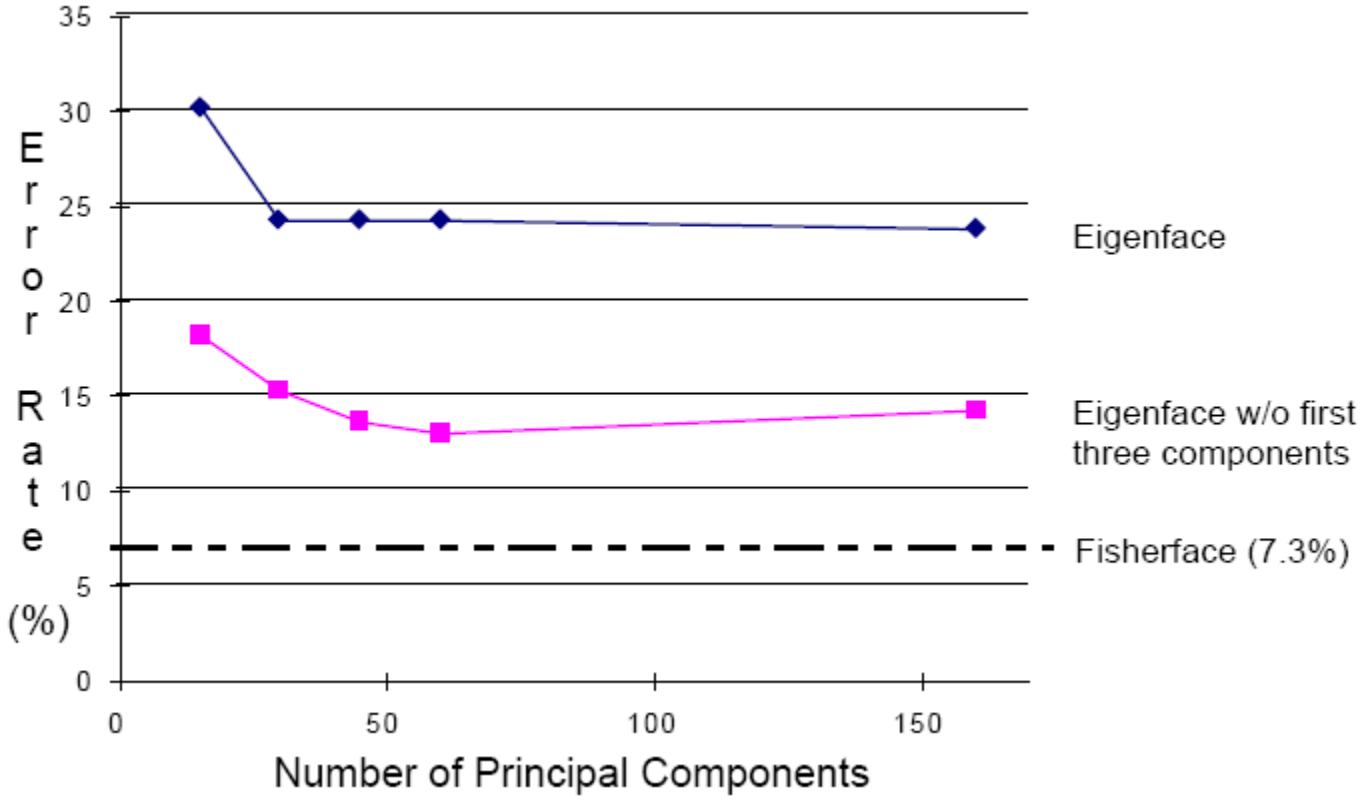
- Input: 160 images of 16 people
- Train: 159 images
- Test: 1 image

- Variation in Facial Expression, Eyewear, and Lighting



Reference: Eigenfaces vs. Fisherfaces, Belheumer et al., PAMI 1997

Eigenfaces vs. Fisherfaces



Reference: Eigenfaces vs. Fisherfaces, Belheumer et al., PAMI 1997

Thank you!